Extração e Preparação de Dados

Aula 13 – Intro a Pipelines

ibmec

Quem sou eu?

Professor: Luís Aramis dos Reis Pinheiro.

- **Doutorado e Mestrado** em **Ciências Mecânicas** – **UnB** – **CAPES 7**
- **Graduação** em Licenciatura em Física – **UNIFAP**

luis.pinheiro@professores.ibemec.edu.br

(96) 99907-5819

@l_aramis

Nivelando o Campo: Feature Scaling

O Objetivo

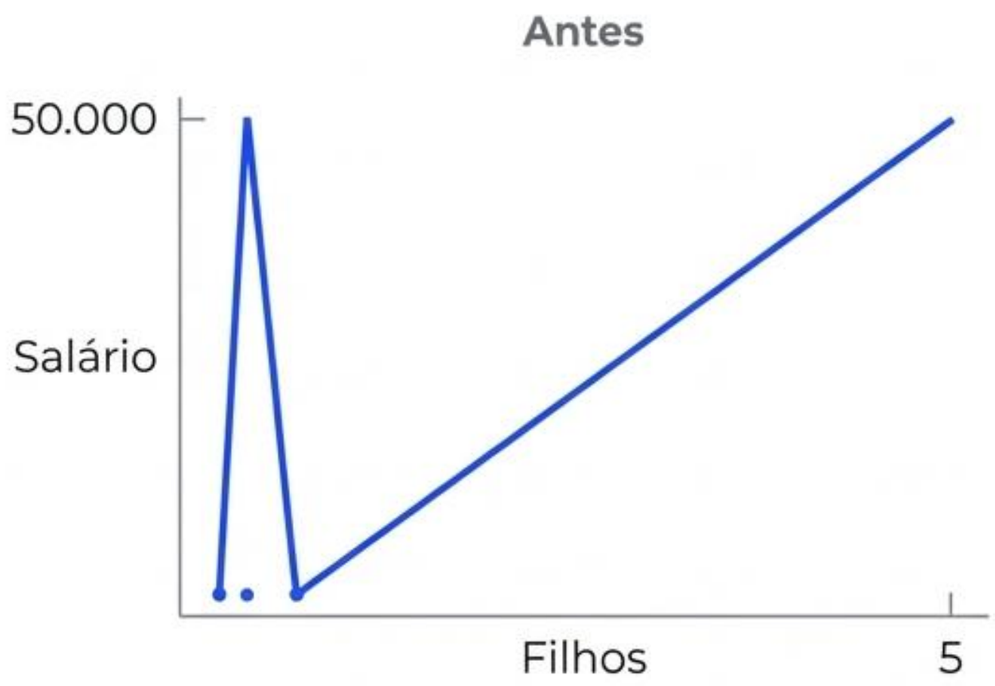
Comprimir todas as variáveis para a mesma escala sem perder a distribuição estatística.

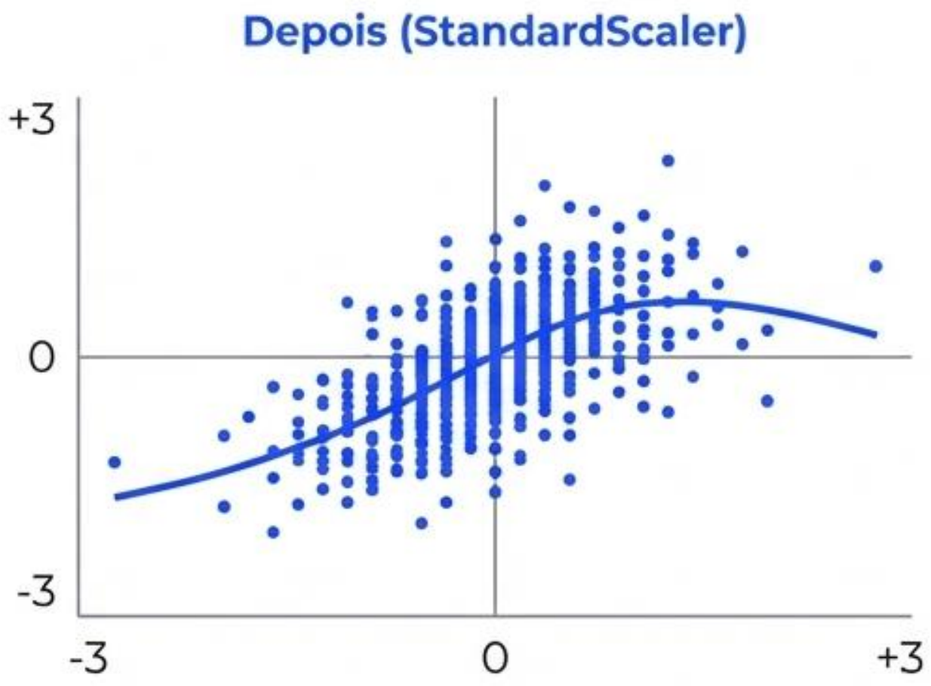
Padronização (StandardScaler)

Centraliza os dados na média 0 e desvio padrão 1 (ideal para dados normais).

Normalização (MinmaxScaler)

Espreme rigorosamente os dados num limite fixo entre 0 e 1.





Insight: Salário e número de filhos passam a falar a mesma língua matemática. Ninguém mais esmaga ninguém.

guem.

Dados Sintéticos: SMOTE

Biblioteca Alvo

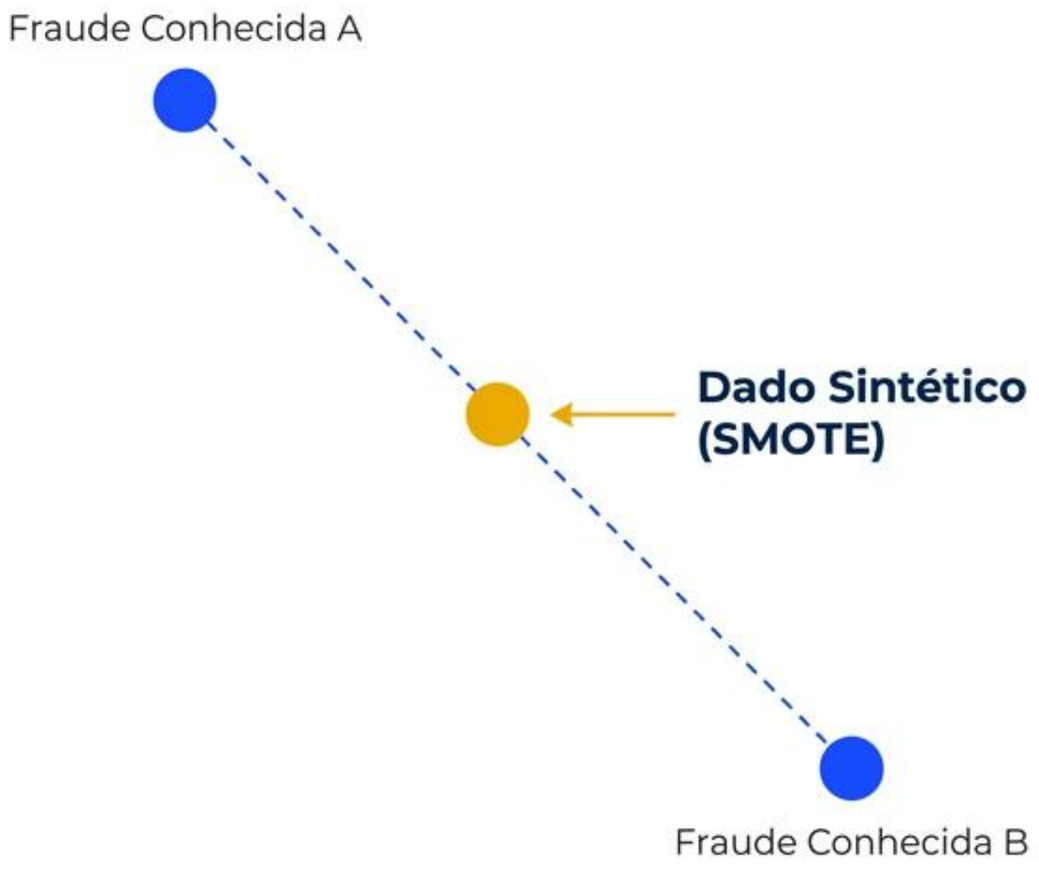
Utilização da biblioteca imbalanced-learn no ecossistema Python.

O que é o SMOTE

Synthetic Minority Over-sampling Technique. Criação matemática de novos dados, evitando o simples copiar e colar.

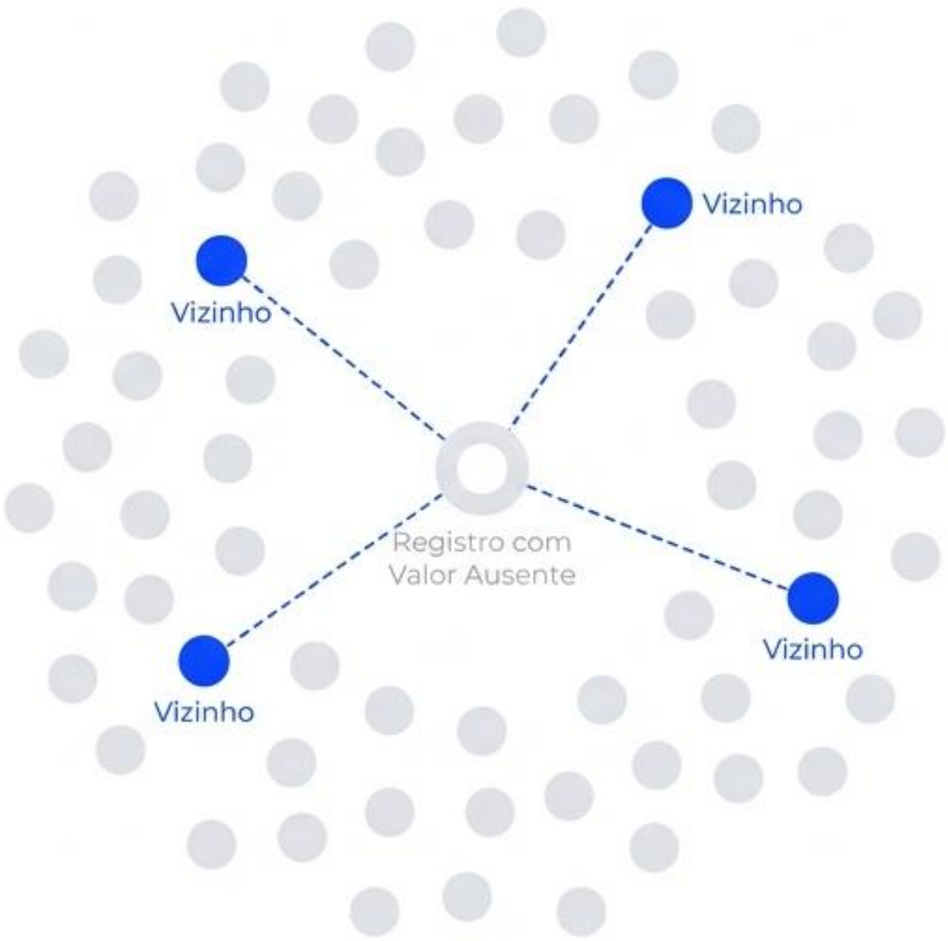
A Mecânica

Em vez de apenas clonar linhas (o que gera overfitting), o algoritmo cria novos dados sintéticos interpolando vizinhos da classe minoritária.



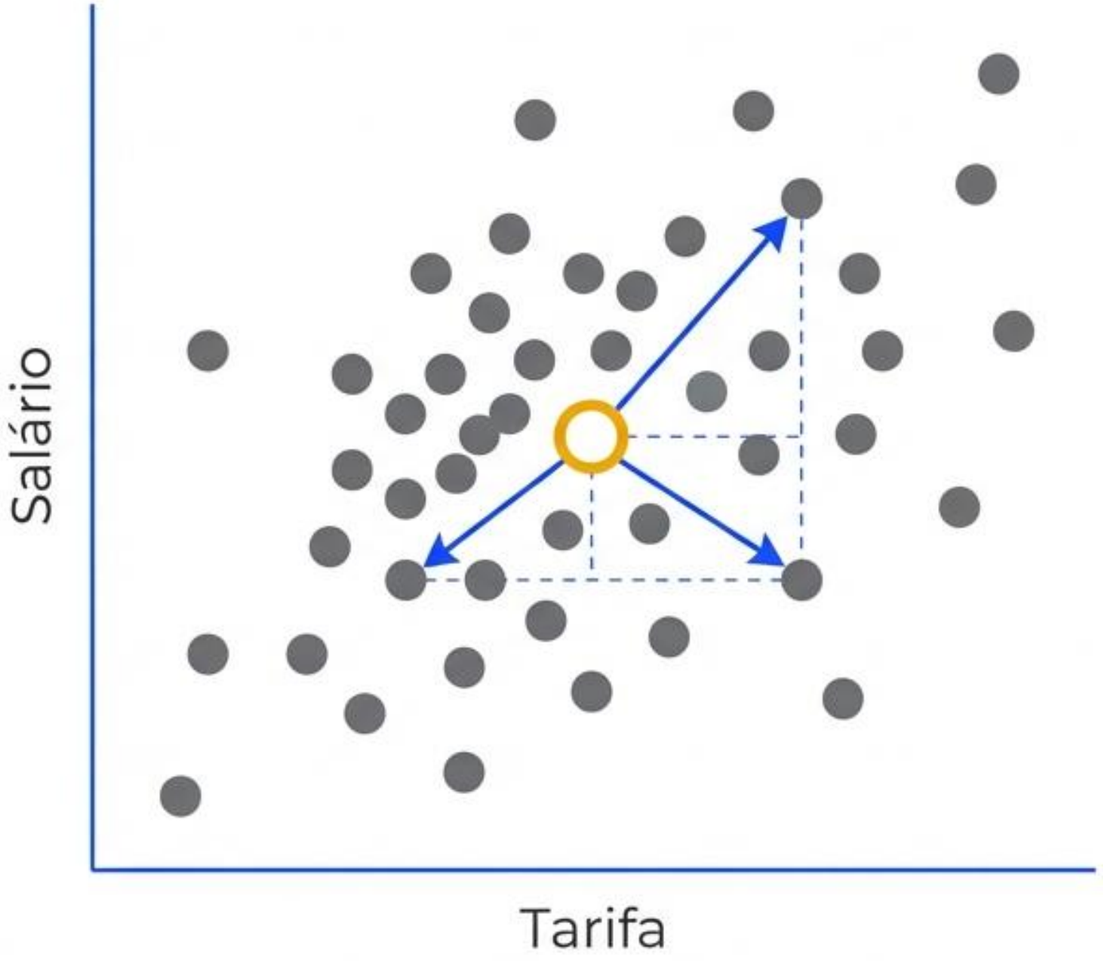
O Algoritmo K-Nearest Neighbors (KNN)

- **K:** O número predefinido de vizinhos a serem consultados.
- **Nearest:** A busca pelos registros mais similares no espaço de dados.
- A imputação ocorre pela média estrita destes K vizinhos idênticos.



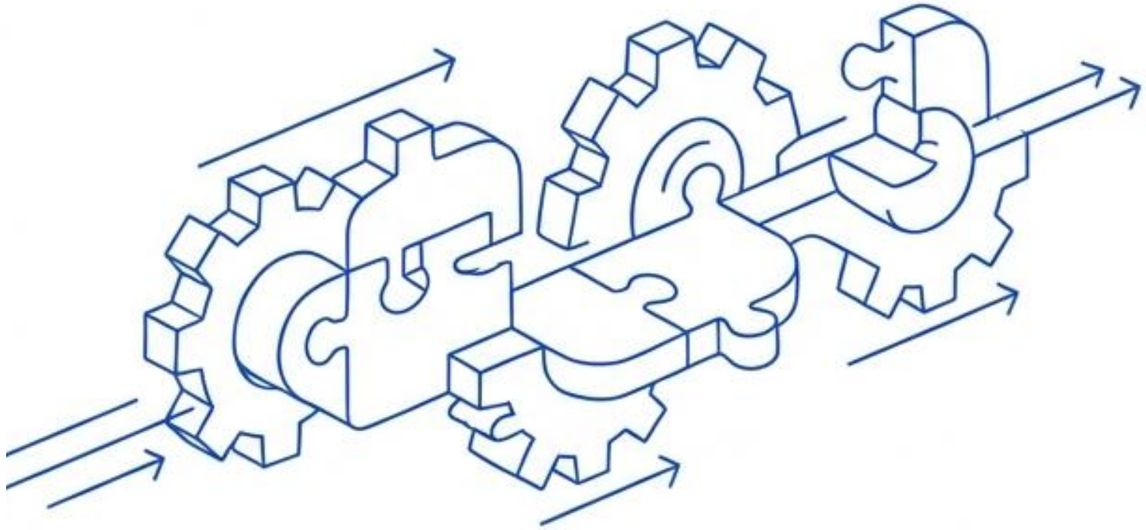
A Matemática da Vizinhança

- Cálculo de distâncias espaciais (ex: Distância Euclidiana).
- Dados transformados em coordenadas geométricas multi-eixos.
- A proximidade física no gráfico traduz a similaridade no negócio.



Transformando Notebooks: Intro a Pipelines

- A Era da Automação no Scikit-Learn
- Disciplina: **Extração e Preparação de Dados (IBM8915)**
- Data: 31/03 (Terça-feira)



O Código Espaguete (O Problema)

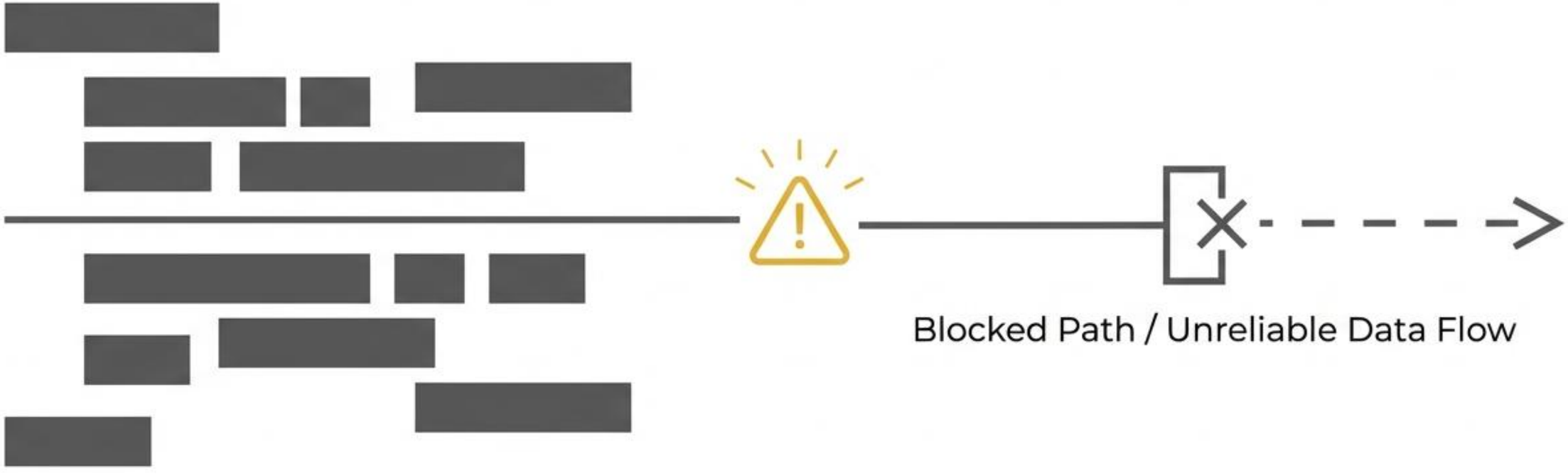
- **A Dificuldade de Manutenção:** dezenas de **passos manuais** são impossíveis de sustentar em produção.
- **O Desperdício de Variáveis:** múltiplos *DataFrames* intermediários **destrói a memória RAM** e a legibilidade.
- **O Risco Letal:** Executar células fora de ordem é a receita perfeita para o ***Data Leakage*** (Vazamento de Dados).

```python
df2 = df1.dropna(subset=['target_comprou'])

X_raw = df2.drop('target_comprou', axis=1)
y = df2['target_comprou']

X_train_raw, X_test_raw, y_train, y_test = train_test_split(X_raw, y, test_size=0.2, random_state=42)

# --- SEPARANDO VARIÁVEIS NUMÉRICAS E CATEGÓRICAS ---
X_train_num = X_train_raw[['idade', 'salario']].copy()
X_test_num = X_test_raw[['idade', 'salario']].copy()

X_train_cat = X_train_raw[['cidade']].copy()
X_test_cat = X_test_raw[['cidade']].copy()

# --- IMPUTAÇÃO DE VALORES NULOS ---
imputer = SimpleImputer(strategy='median')

X_train_imputed = imputer.fit_transform(X_train_num)
X_test_imputed = imputer.transform(X_test_num)

# Transformar de volta para DataFrame
X_train_num_df = pd.DataFrame(X_train_imputed, columns=['idade', 'salario'], index=X_train_raw.index)
X_test_num_df = pd.DataFrame(X_test_imputed, columns=['idade', 'salario'], index=X_test_raw.index)

# --- ONE HOT ENCODING ---
encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')

X_train_encoded = encoder.fit_transform(X_train_cat)
X_test_encoded = encoder.transform(X_test_cat)

cat_columns = encoder.get_feature_names_out(['cidade'])

# Transformar de volta para DataFrame
X_train_cat_df = pd.DataFrame(X_train_encoded, columns=cat_columns, index=X_train_raw.index)
```

Python

A Vulnerabilidade do Código Solto

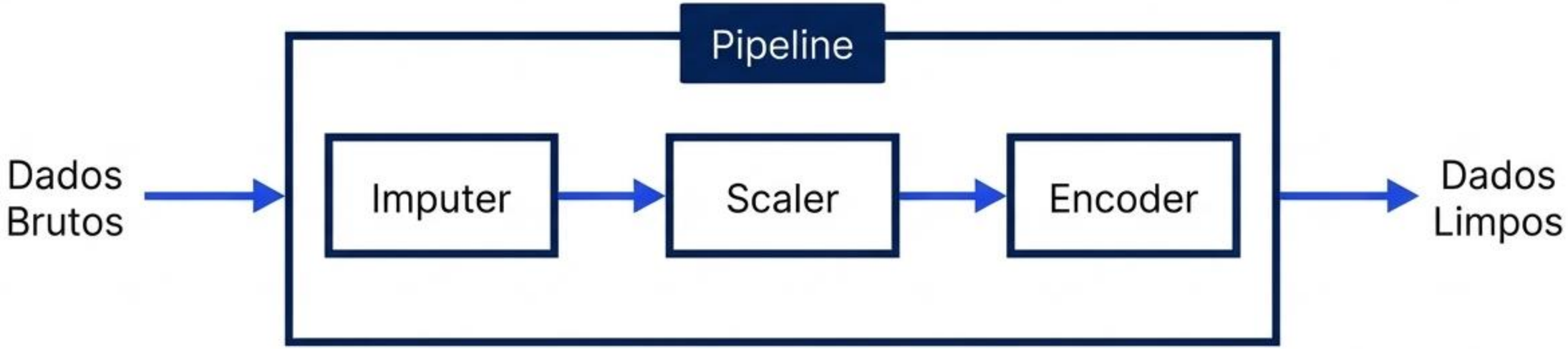
- Repetição exaustiva de código entre bases de treino e teste.
- Alta complexidade para manutenção e versionamento em produção.
- Risco crítico de **Data Leakage** (Vazamento de Dados).



O Paradigma do Pipeline

- Introdução ao subpacote **sklearn.pipeline**.
- Encapsulamento de múltiplas transformações em um único objeto.
- Garantia de execução sequencial e proteção contra vazamentos.

Classe



O Paradigma do Pipeline

- **O Conceito de Pipeline:** Um contêiner que encadeia múltiplos passos de processamento sequencialmente.
- **A Abstração:** Transforma uma rotina caótica em um único objeto coerente.
- **O Encadeamento:** A saída de um passo (*Transformador*) torna-se automaticamente a entrada do próximo.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
```

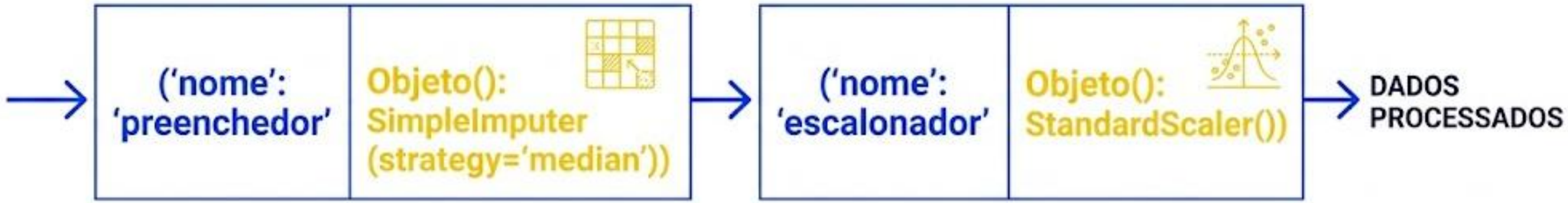
A Anatomia de um Pipeline (As Tuplas)

- **A Regra da Tupla:** Cada passo é definido por ('nome_do_passo', Transformador()).
- **Nomenclatura Livre:** O primeiro elemento da tupla é uma *string* escolhida por você para facilitar a identificação e o *debug* no futuro.
- **A Ordem Importa:** O fluxo é estritamente sequencial, rodando de cima para baixo.

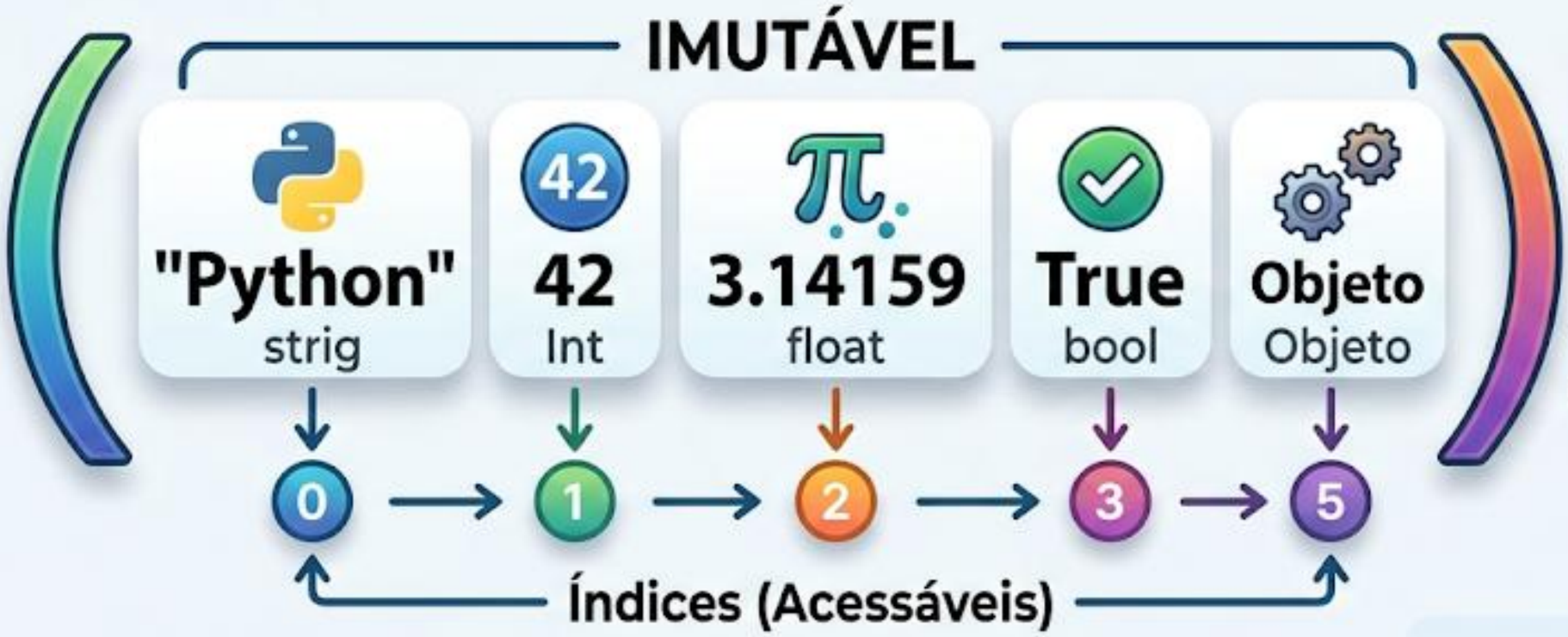
```
# A estrutura base: Lista de Tuplas ('nome', Objeto())
meu_fluxo = Pipeline(steps=[
    ('preenchedor', SimpleImputer(strategy='median')),
    ('escalonador', StandardScaler())
])
```

A Anatomia de um Pipeline (As Tuplas)

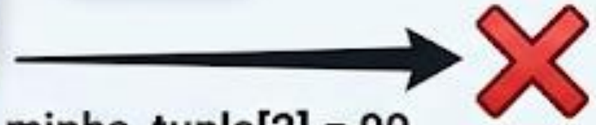
```
# A estrutura base: Lista de Tuplas ('nome', Objeto())
meu_fluxo = Pipeline(steps=[
    ('preenchedor', SimpleImputer(strategy='median')),
    ('escalonador', StandardScaler())
])
```



O que é uma TUPLA?



Tuplas não podem ser alteradas após a criação



minha_tupla[2] = 99

ERRO: Não é permitido modificar

```
minha_tupla = ("Python", 42, 3.14, True, {...})

# Tentativa de alteração (gera erro):
minha_tupla[1] = 100 # ❌ ERRO: Tuplas são imutáveis!
```

SÍNTESE VISUAL

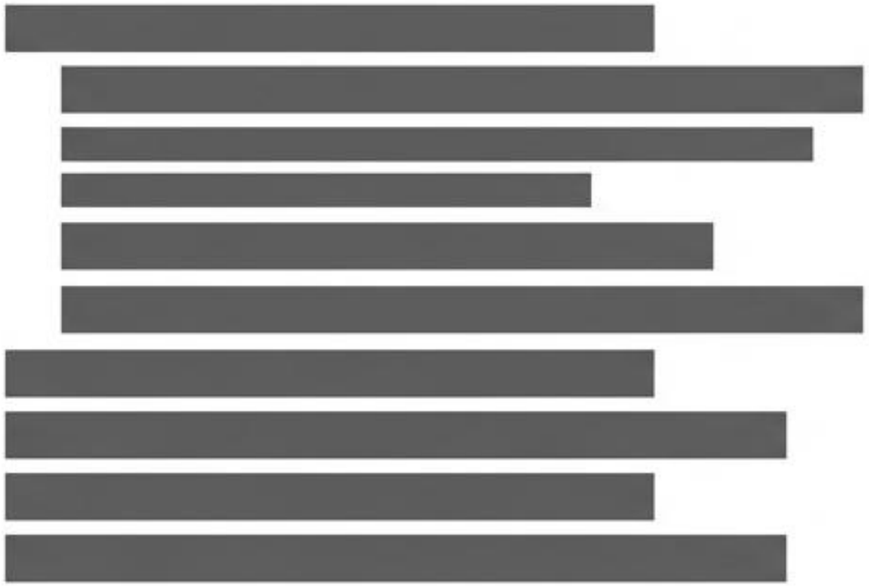
TUPLA imutável
parênteses ( )

LISTA mutável
colchetes [ ]

Mentalidade de Engenharia de Dados

- Transição de análise procedural para Orientação a Objetos.
- Redução de dezenas de linhas de código para apenas 3 ou 4.
- Construção do pré-requisito para orquestradores (Apache Airflow).

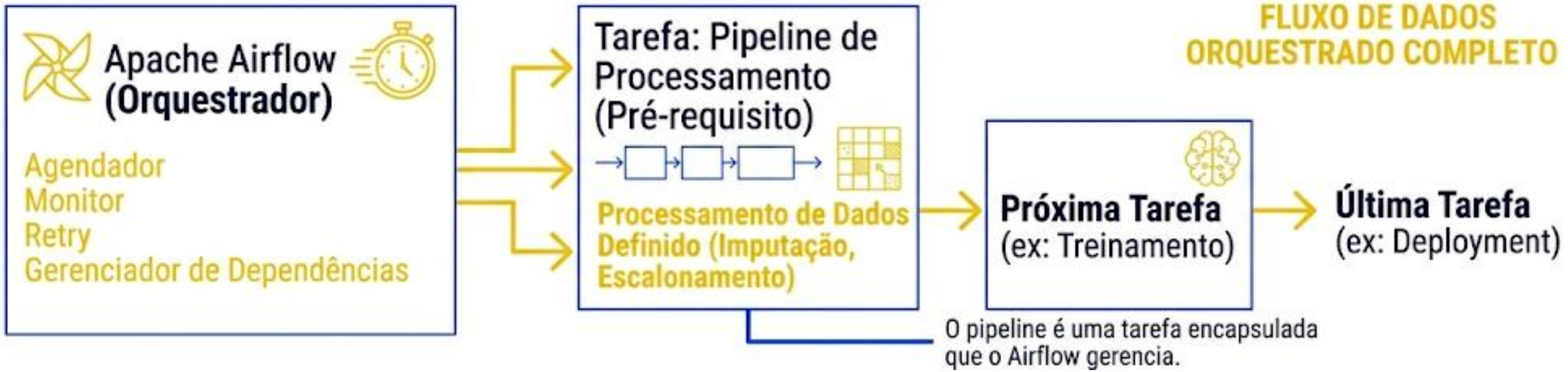
Antes (Procedural) ✗



Depois (Pipeline) ✓

O Orquestrador: Apache Airflow

O que é e qual pré-requisito ele preenche.



A Mágica do .fit_transform() Sequencial

 # O código de 50 linhas vira apenas 1 linha!

```
dados_processados =
meu_fluxo.fit_transform(X_treino)
```

A Execução em Cadeia:

Ao chamar o método, o Pipeline executa o **.fit_transform()** no Passo 1, pega a matriz resultante e passa invisivelmente para o **.fit_transform()** do Passo 2.

Isolamento Absoluto:

Nenhuma variável intermediária é criada na memória global do seu computador.

O Paradigma da Segurança: Prevenção de Data Leakage

- **Data Leakage (Vazamento de Dados):** Ocorre quando o modelo 'espia' informações do futuro (Teste) durante a fase de adequação matemática.
- **A vulnerabilidade do código manual:** O recálculo acidental de premissas em variáveis soltas.
- **O Escudo do Pipeline:** O método .fit() retém o estado (ex: média salarial) exclusivamente na base de Treino.
- **Garantia de Engenharia:** No Teste, o Scikit-Learn aplica as exatas mesmas constantes do passado de forma automática.

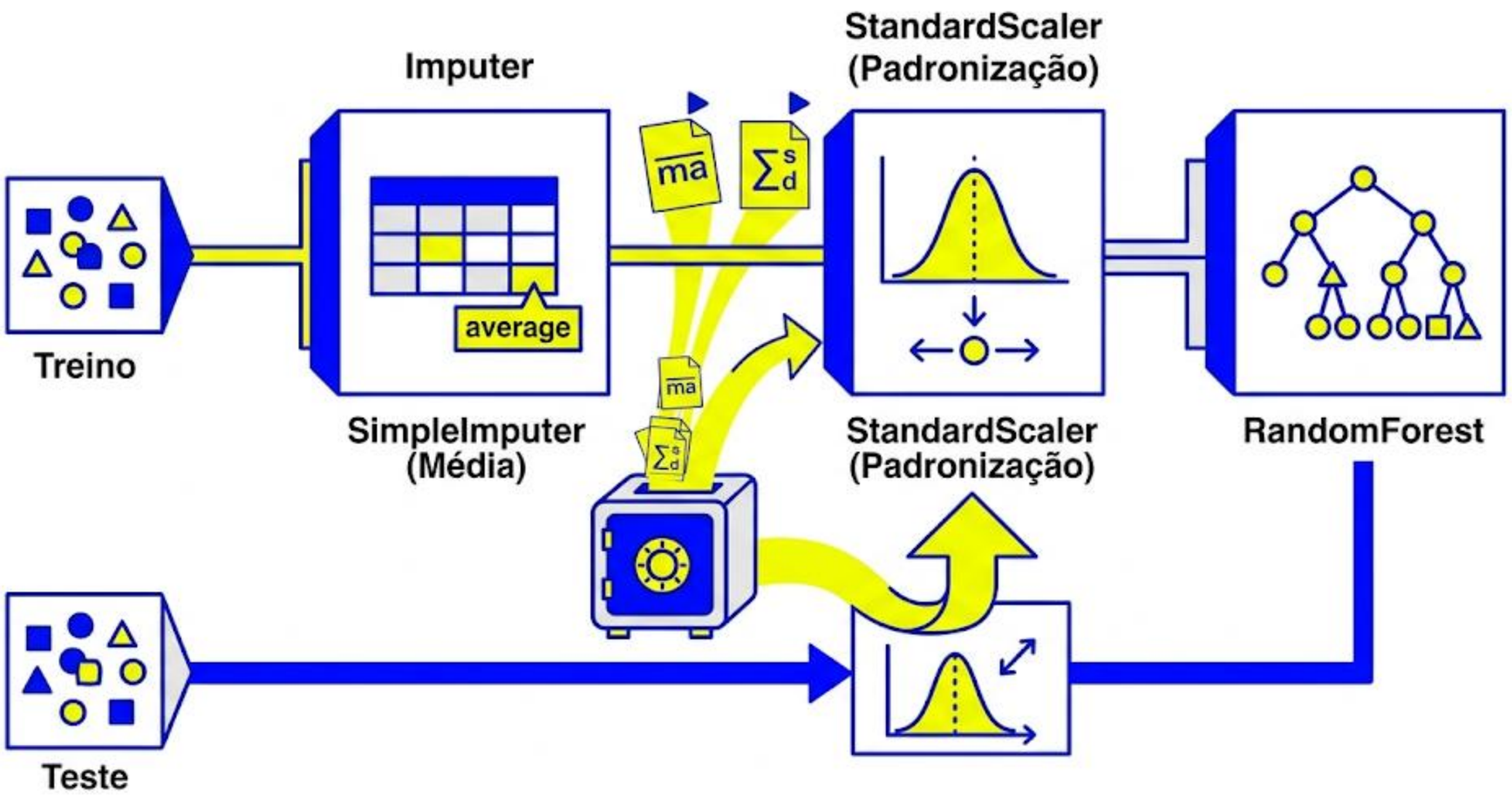
O Paradigma da Segurança: Prevenção de Data Leakage

```
# A execução segura no mundo real
modelo_treinado = meu_fluxo.fit(X_treino, y_treino)

# O teste JAMAIS recalcula médias ou medianas
previsoes = meu_fluxo.transform(X_teste)
```

Check de Aprendizado (O Comportamento do Fluxo)

- **Cenário:** Você criou um `Pipeline` com 3 passos: `SimpleImputer` (Média), `StandardScaler` (Padronização) e `RandomForest` (Modelo Preditivo). Você executa a instrução `meu_fluxo.transform(X_teste)`.
- **Pergunta:** "De onde o `StandardScaler` vai tirar a *Média* e o *Desvio Padrão* que ele usará para escalonar essa matriz de teste que acabou de chegar?"

Da base de Treino! O Pipeline armazenou internamente a média salarial e o desvio padrão da base de Treino durante a fase do .fit(). No Teste, o Scikit-Learn vai aplicar as mesmas constantes da semana passada. Esse é o nível de segurança que a engenharia de software exige.



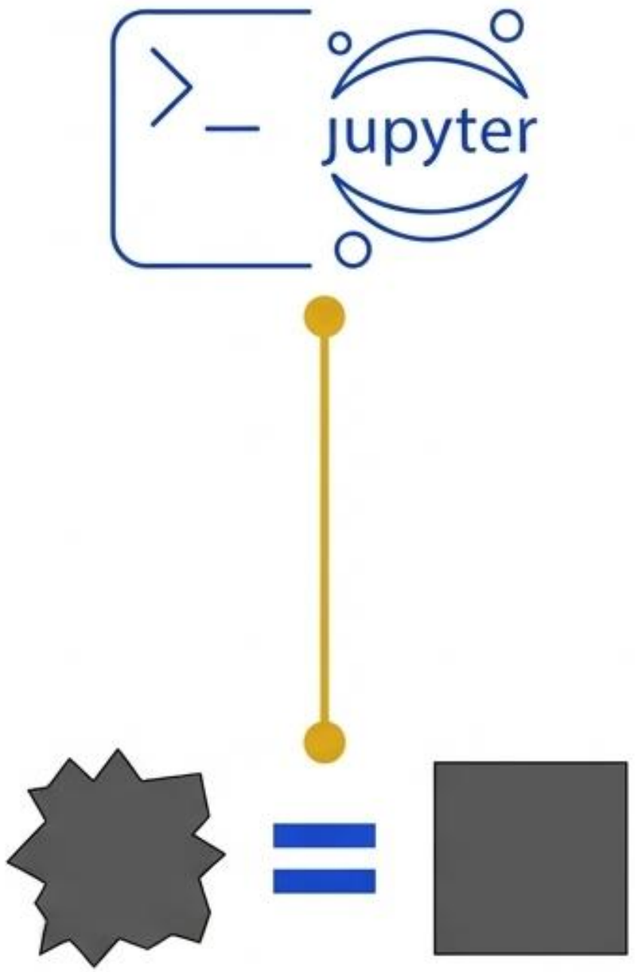
Laboratório: O Fluxo Reprodutível

Missão:
Abrir o notebook **lab_13_pipelines_intro.ipynb**

Ação 1:
Refatorar as rotinas de limpeza das aulas anteriores.

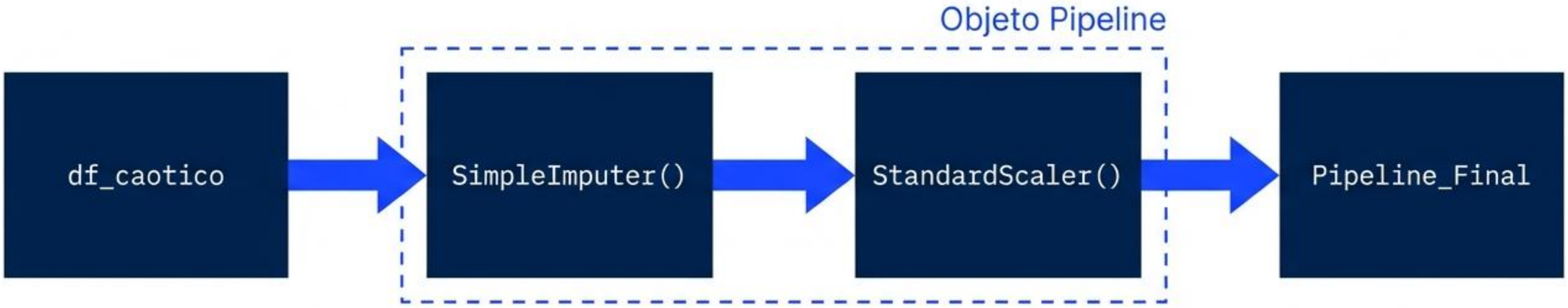
Ação 2:
Combinar imputação e scaling via Pipeline.

Validação:
Comprovar a reprodutibilidade da matriz final.



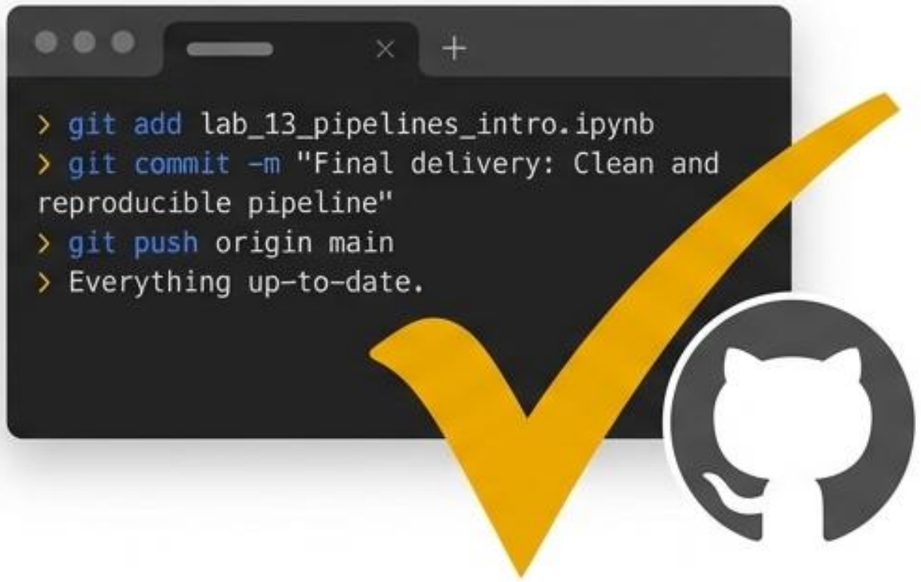
Hands-on: O Arquiteto de Pipelines (Lab 13)

- **Análise do Caos:** Examinar o código legado repleto de variáveis temporárias e imputações esparsas.
- **Montagem de Componentes:** Instanciar isoladamente o SimpleImputer e o StandardScaler.
- **Encadeamento Estrutural:** Construir a classe Pipeline passando a lista de tuplas ('nome', Transformador).
- **Execução Limpa:** Rodar o fluxo contínuo do imputador ao scaler sem intervenções manuais.



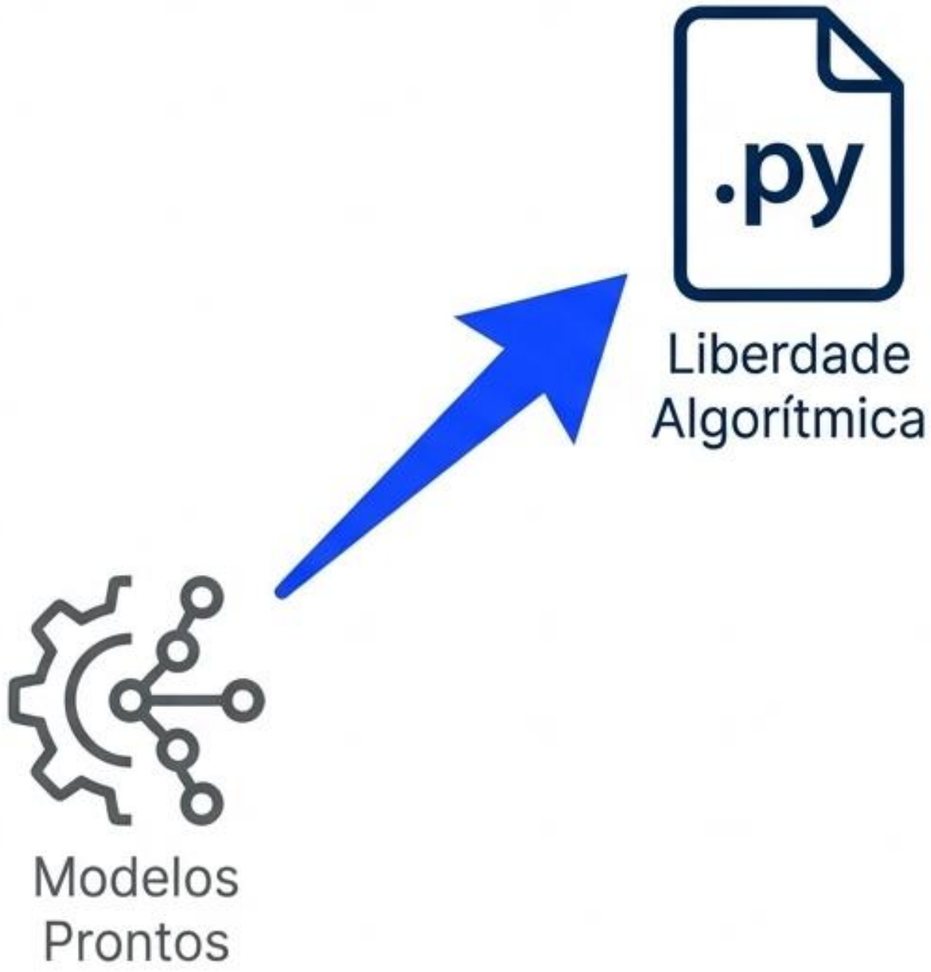
Padrão de Engenharia: O Entregável do Laboratório

- **Entrega**: Notebook **lab_13_pipelines_intro.ipynb** limpo e 100% reprodutível.
- **Regra de Aprovação (Tolerância Zero)**: Nenhuma modificação manual ou variável temporária solta no fluxo de dados de treino.
- **Encapsulamento Absoluto**: Toda a lógica matemática deve estar estritamente dentro da classe Pipeline.
- **Versionamento**: Executar o push direto para a branch principal (main) do repositório.



O Salto para a Engenharia Pura (Próxima Aula)

- **A limitação dos transformadores nativos:** O que fazer quando a lógica de negócio não existe no Scikit-Learn?
- **Orientação a Objetos:** Transição da análise de scripts para arquitetura de Software.
- **Custom Transformers:** Custom Transformers: Criação de classes próprias em Python puro.
- **Preparação de Infraestrutura:** O último degrau antes da orquestração com Apache Airflow.



Dúvidas?

CRIATIVIDADE
INOVAÇÃO
¡DEIA!
COMAMUNIDADE
CRIATIVIDADE
BRAINSTORM
INOVAÇÃO

ibmec

A direção do amanhã.

/ibmec